

회원 관리 프로그램 — TRD (기술 요구사항 정의서)

항목	내용
문서명	회원 관리 프로그램 TRD
문서 유형	Technical Requirements Document
버전	v1.0
작성일	2026-06-03
대상 언어	Python 3.10 이상
의존성	표준 라이브러리만 사용 (외부 패키지 없음)
상태	확정(Baseline)

개정 이력

버전	일자	변경 내용
v1.0	2026-06-03	최초 작성

TRD란?(안내) PRD가 "무엇을 만들까"라면, TRD는 "어떻게 만들까" 를 적는 기술 설계 문서다. 아키텍처 (구조), 모듈/함수 설계, 데이터 구조, 사용할 기술과 버전, 오류 처리 방법을 담는다.

1. 기술 스택 및 버전 (Tech Stack)

구성요소	선택	버전 기준	비고
언어	Python	3.10 이상 권장	2026-06 기준 최신 안정판은 3.14.x, 직전 라인 3.13.x
직렬화(저장)	<code>pickle</code>	표준 라이브러리	바이너리 파일 저장/복원
유효성 검사	<code>re</code> (정규식)	표준 라이브러리	전화번호 형식 검증
실행 환경	콘솔/터미널	OS 무관	Windows/macOS/Linux 공통

1-1. 라이브러리 버전 충돌 분석 (요청 반영)

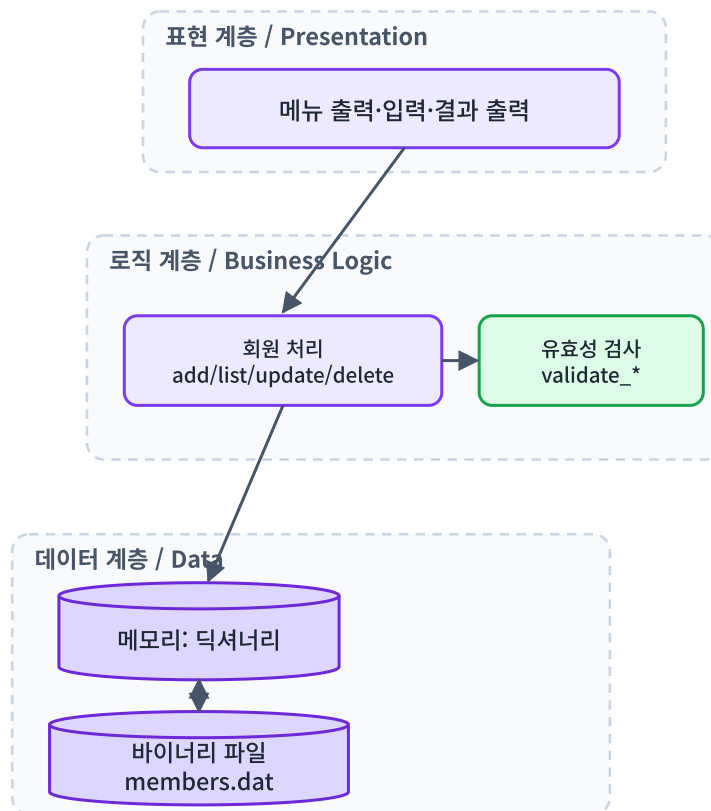
결론: 버전 충돌 위험 없음. 본 프로그램은 `pickle`, `re` 등 Python 표준 라이브러리만 사용하고 외부 패키지(`pip install` 대상)를 전혀 사용하지 않는다. 표준 라이브러리는 Python 인터프리터에 내장되어 별도 버전을 고정·관리할 필요가 없으므로, 패키지 간 의존성 충돌(dependency conflict)이 원천적으로 발생하지 않는다.

점검 항목	결과
외부 패키지 의존성	없음
<code>requirements.txt</code> 필요 여부	불필요
버전 핀(pin) 관리 필요	불필요
권장 최소 버전	Python 3.10 (<code>match</code> 문 등 가독성 향상 기능 활용 가능)
하위 호환	<code>pickle</code> / <code>re</code> 는 Python 3 전 버전에서 동작 → 3.8 이상이면 무난

실무 메모: 실제 프로젝트라면 외부 패키지를 쓸 때 `requirements.txt` 또는 `pyproject.toml` 로 버전을 고정하고, 가상환경(venv/conda)에서 충돌을 검증한다. 이 과제는 그 단계가 불필요한 가장 단순한 형태다.

2. 시스템 아키텍처 (Architecture)

콘솔 프로그램이지만 **3계층(Layered)** 구조로 설계하면 가독성과 유지보수성이 좋아진다.



계층	책임	해당 함수(예)
표현 계층	사용자와 입출력	<code>print_menu</code> , <code>input(...)</code>
로직 계층	검증·CRUD 처리	<code>validate_phone</code> , <code>add_member</code> , <code>update_member</code>
데이터 계층	메모리·파일 보관	<code>load_data</code> , <code>save_data</code>

핵심 포인트: "화면 그리는 코드", "데이터 다루는 코드", "파일 저장 코드"를 섞지 않고 함수로 나누는 것이 핵심이다. 나중에 화면을 GUI로 바꿔도 로직/데이터 계층은 그대로 재사용할 수 있다.

3. 데이터 구조 설계 (Data Structure)

3-1. 회원 1명 (딕셔너리)

```
member = {
    "name": "윤아",          # 이름 (str, 1~5자)
    "phone": "01012345678", # 전화번호 (str, 식별자)
    "addr": "서울시",       # 주소 (str)
    "type": "친구",         # 종류 (str: 가족/친구/기타)
}
```

3-2. 전체 회원 컬렉션 — 두 가지 설계안

원본 과제는 "딕셔너리 처리"를 요구한다. 컬렉션 구조는 두 방식이 가능하다.

설계안	구조	장점	단점
A. <code>dict[전화번호] = member</code> (권장)	<code>{ "01012345678": {member...}, ... }</code>	전화번호로 즉시 조회($O(1)$), 식별자 명확	이름 검색은 전체 순회 필요
B. <code>list[member]</code>	<code>[{member...}, {member...}]</code>	가장 직관적, 입력 순서 유지	식별자 개념이 약함

권장: 설계안 A. 동명이인이 존재하지만 전화번호는 고유하므로, 전화번호를 key로 두면 "같은 회원 중복 등록"을 자연스럽게 막을 수 있다. 이름 검색(수정/삭제)은 값들을 순회하여 이름이 일치하는 항목을 모은다.

```
# 설계안 A 예시
members = {} # 시작 시 빈 딕셔너리
members["01012345678"] = {"name": "윤아", "phone": "01012345678", "addr": "서울시", "type": ...}

# 이름으로 검색 (동명이인 → 리스트로 수집)
hits = [m for m in members.values() if m["name"] == "윤아"]
```

4. 모듈 / 함수 설계 (Function Spec)

단일 파일(`member_manager.py`)에 기능별 함수로 분리한다.

함수	입력	출력/효과	설명
<code>load_data(path)</code>	파일 경로	dict	파일 읽어 복원, 없으면 빈 dict
<code>save_data(path, members)</code>	경로, dict	(파일 기록)	pickle로 바이너리 저장
<code>print_menu()</code>	-	(화면 출력)	메뉴 출력
<code>validate_name(name)</code>	str	bool	1~5자 검사
<code>validate_phone(phone)</code>	str	bool	정규식 형식 검사
<code>validate_type(t)</code>	str	bool	가족/친구/기타 검사
<code>input_member()</code>	-	dict	검증 통과한 회원 정보 입력 받기
<code>add_member(members)</code>	dict	(dict 갱신)	회원 추가
<code>list_members(members)</code>	dict	(화면 출력)	총원 + 목록 출력
<code>find_by_name(members, name)</code>	dict, str	list	이름 일치 회원 목록
<code>update_member(members)</code>	dict	(dict 갱신)	검색→선택→수정
<code>delete_member(members)</code>	dict	(dict 갱신)	검색→선택→삭제
<code>main()</code>	-	-	전체 루프 제어

5. 영속화 설계 (Persistence with pickle)

`pickle`은 파이썬 객체(딕셔너리 등)를 **바이너리로 변환(직렬화)** 하여 파일에 저장하고, 다시 읽어 **원래 객체로 복원(역직렬화)** 한다.

```
import pickle

# 저장 (직렬화) - 반드시 바이너리 모드 'wb'
def save_data(path, members):
    with open(path, "wb") as f:
        pickle.dump(members, f)

# 읽기 (역직렬화) - 바이너리 모드 'rb', 파일 없으면 빈 dict
def load_data(path):
    try:
        with open(path, "rb") as f:
            return pickle.load(f)
    except FileNotFoundError:
        return {} # 최초 실행
    except (pickle.UnpicklingError, EOFError):
        return {} # 파일 손상 시 안전 복구
```

핵심 포인트: 텍스트 파일은 'w' / 'r', 바이너리 파일은 'wb' / 'rb' 를 쓴다. pickle은 바이너리이므로 반드시 **b** 가 붙은 모드를 사용해야 한다.

5-1. ⚠ pickle 보안 주의(실무 관점)

pickle 은 신뢰할 수 없는 출처의 파일을 역직렬화하면 임의 코드가 실행될 위험이 있다(이는 Python 공식 문서가 경고하는 사항이다). 본 과제처럼 **내 프로그램이 직접 만든 파일만** 읽는 경우는 안전하다. 다만 실무에서 외부에서 받은 pickle 파일을 그냥 **load** 하면 안 된다. 실무에서는 사용자 데이터 교환에 JSON을, 신뢰 데이터 보존에 pickle을 쓰는 식으로 구분한다.

5-2. 저장 시점 전략

전략	설명	권장 상황
종료 시 저장 (원본 기준)	메뉴 5에서 1회 저장	과제 요구사항 충족
즉시 저장 (대안)	추가/수정/삭제 직후마다 저장	비정상 종료에도 데이터 보존 (실무 권장)

6. 유효성 검사 기술 명세 (Validation)

```
import re

def validate_name(name: str) -> bool:
    return 1 <= len(name) <= 5          # 이름 1~5자

def validate_phone(phone: str) -> bool:
    # 권장안: 하이픈 없는 11자리 (010으로 시작) - 화면 캡처 기준
    return re.fullmatch(r"010\d{8}", phone) is not None
    # 대안(하이픈 포함 규칙 텍스트 기준):
    # return re.fullmatch(r"\d{3}-\d{4}-\d{4}", phone) is not None

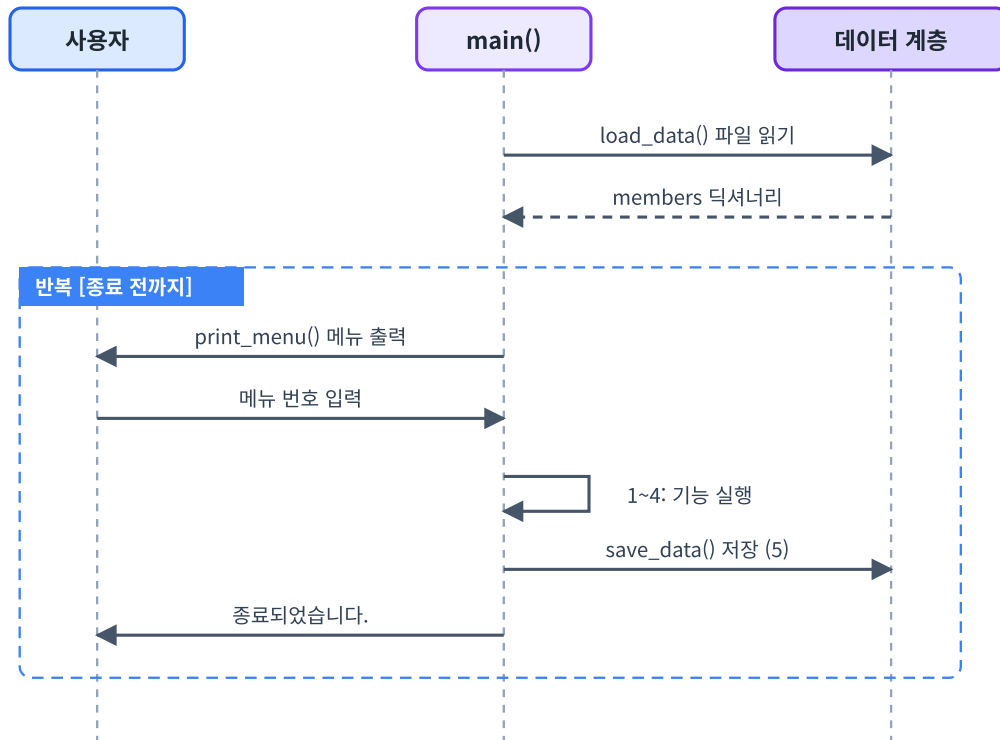
def validate_type(t: str) -> bool:
    return t in ("가족", "친구", "기타")  # 셋 중 하나만
```

정규식	의미
<code>010\d{8}</code>	<code>010</code> + 숫자 8자리 = 총 11자리
<code>\d{3}-\d{4}-\d{4}</code>	숫자3 - 숫자4 - 숫자4 (하이픈 포함)
<code>\d</code>	숫자 한 글자 / <code>{n}</code>

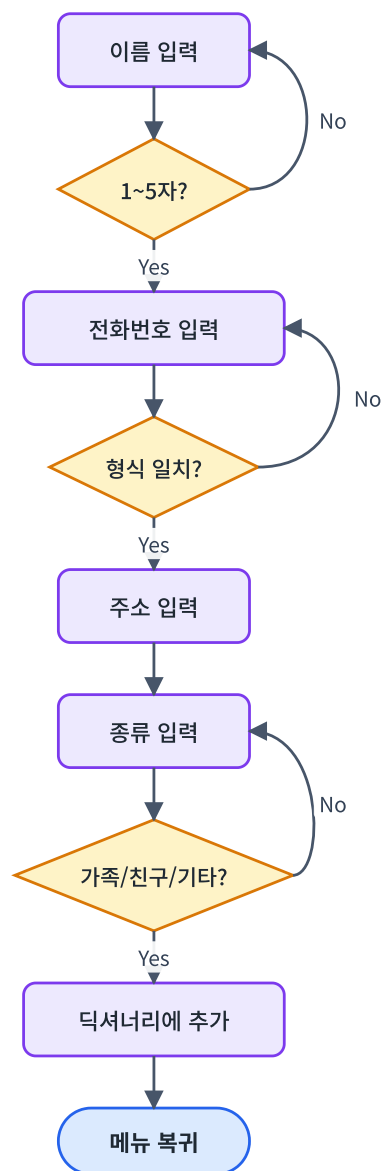
전화번호 형식은 구현요구사항 § 4의 미해결 이슈다. 코드에 둘 다 주석으로 남겼으니, 수업에서 확정 후 한 쪽만 활성화한다.

7. 단계별 워크플로우 (Step-by-Step Workflow)

7-1. 프로그램 메인 루프



7-2. 회원 추가(add_member) 상세



8. 에러/예외 처리 전략 (Error Handling)

위치	예외	처리 코드 패턴
파일 읽기	<code>FileNotFoundError</code>	빈 dict 반환
파일 읽기	<code>EOFError</code> , <code>UnpicklingError</code>	빈 dict 반환(손상 복구)
메뉴 입력	숫자 변환 실패	입력을 문자열로 받아 비교, 또는 <code>try/except ValueError</code>
번호 선택	범위 밖/문자	재입력 루프
검색	결과 0건	"해당하는 회원 정보가 없습니다." 후 복귀


```
# 메뉴 입력 안전 처리 예시
choice = input("선택: ").strip()
if choice not in ("1", "2", "3", "4", "5"):
    print("잘못된 입력입니다.")
    continue
```

원칙(NFR-01 연계): 어떤 입력에도 프로그램이 죽지 않도록, 입력을 받는 모든 지점에 검증 또는 `try/except` 를 둔다.

9. 테스트 시나리오 (Test Cases)

TC	입력/행위	기대 결과
TC-01	최초 실행(파일 없음)	오류 없이 빈 목록으로 시작
TC-02	정상 회원 추가 후 목록 보기	추가한 회원이 보임, 총원 +1
TC-03	이름 6자 입력	거부, 재입력 요구
TC-04	전화번호 123 입력	거부, 재입력 요구
TC-05	종류 동료 입력	거부, 재입력 요구
TC-06	동명이인 2명 중 1명 수정	번호 선택 후 해당 회원만 수정
TC-07	없는 이름 삭제 시도	"해당하는 회원 정보가 없습니다."
TC-08	메뉴에 abc 입력	"잘못된 입력입니다.", 미종료
TC-09	종료 후 재실행	직전 데이터 그대로 복원

10. 디렉터리 구조 (제안)

```
member_manager/
├── member_manager.py    # 메인 프로그램 (단일 파일)
├── members.dat          # 자동 생성되는 바이너리 데이터 파일
└── README.md           # 실행 방법 안내
```

실행: 터미널에서 `python member_manager.py`

11. 구현 체크리스트 (개발자용)

- [] `load_data` / `save_data` 바이너리 모드(`rb` / `wb`) 확인
- [] 시작 시 `load_data` , 종료 시 `save_data` 호출
- [] 4개 CRUD 함수 분리 구현
- [] 3종 유효성 함수(`validate_name/phone/type`)
- [] 동명이인 번호 선택 로직
- [] 없는 회원 / 잘못된 메뉴 예외 처리
- [] 한글 출력 인코딩 정상 확인
- [] 테스트 시나리오 TC-01~09 통과